

SPENDDNA - Your Wallet's Year-End Story

Week 2 Minor Project - The Unlax Academy

This notebook analyzes Rahul Sharma's UPI/Bank transactions (Jan-Jun 2024)

Built for Colab - Run all cells sequentially

Done by - SHOYAL HALDAR

Date : 3-july-2026

✓ SpendDNA: Spotify Wrapped for Your Money 💰

Analyzing 6 months of Rahul Sharma's transactions

Built as part of The Unlax Academy Week 2 Minor Project

Features Implemented:

- Transaction Parser
- Vendor Extractor
- Category Tagger
- Spending Overview
- Monthly Trends
- Time-of-Day Patterns
- Anomaly Detection
- Spending Archetypes

Imports

```
1 import pandas as pd
2 import numpy as np
3 from datetime import datetime
4
5 print("✅ Libraries imported successfully!")
```

✅ Libraries imported successfully!

✓ Load & Parse Data (Feature 1 - Transaction Parser)

```
1 # Feature 1: Transaction Parser
2 # Feature 1: Transaction Parser
3 df = pd.read_csv('/content/Data set for DADS June.csv')
4 df = df.drop_duplicates().reset_index(drop=True)
5
6 # --- Date: 4 formats mixed in one column (ISO, DD/MM/YY, DD-Mon-YY, DD Mon YYYY) ---
7 # A single pd.to_datetime(dayfirst=True) call can't handle all 4 at once -
8 # it either fails to parse most rows, or worse, silently misreads the
9 # unambiguous ISO rows. So each format is detected by string shape first,
10 # then parsed with its own exact format.
11 s = df['Date'].astype(str).str.strip()
12 is_iso = s.str.len().eq(10) & s.str[4].eq('-') # 2024-01-01
13 is_slash = s.str.contains('/') # 02/01/24
14 is_dmy3 = s.str.contains('-') & ~is_iso # 01-Jan-24
15 is_dmy4 = s.str.contains(' ') # 01 Jan 2024
16
17 parsed = pd.Series(pd.NaT, index=s.index, dtype='datetime64[ns]')
18 parsed[is_iso] = pd.to_datetime(s[is_iso], format='%Y-%m-%d', errors='coerce')
19 parsed[is_slash] = pd.to_datetime(s[is_slash], format='%d/%m/%y', errors='coerce')
20 parsed[is_dmy3] = pd.to_datetime(s[is_dmy3], format='%d-%b-%y', errors='coerce')
21 parsed[is_dmy4] = pd.to_datetime(s[is_dmy4], format='%d %b %Y', errors='coerce')
22 df['Date'] = parsed
23 assert df['Date'].isnull().sum() == 0, "Unparsed dates remain"
```

```

24
25 # --- Amount: strip currency symbols/text and commas ---
26 amt = (df['Amount'].astype(str).str.strip()
27         .str.replace('₹', '', regex=False)
28         .str.replace('Rs.', '', regex=False)
29         .str.replace(',', '', regex=False).str.strip())
30 df['Amount'] = pd.to_numeric(amt, errors='coerce')
31 assert df['Amount'].isnull().sum() == 0, "Unparsed amounts remain"
32
33 # --- Type: standardize to Debit/Credit ---
34 df['Type'] = df['Type'].map({'DR': 'Debit', 'Debit': 'Debit', 'CR': 'Credit'})
35 assert df['Type'].isnull().sum() == 0
36
37 print(f"✅ Parsed {len(df)} transactions")
38 print(f"Date Range: {df['Date'].min().date()} to {df['Date'].max().date()}")

```

✅ Parsed 1310 transactions
Date Range: 2024-01-01 to 2024-06-30

Vendor Extractor (Feature 2)

```

1 #Vendor Extractor (Feature 2)
2
3 # Ai assested : understanding the approch and logic visulization.
4 def extract_vendor(desc):
5     if not isinstance(desc, str): return 'UNCATEGORIZED'
6     d = desc.upper()
7
8     if any(x in d for x in ['INSTAMART', 'KIRANAKART']): return 'INSTAMART'
9
10    if any(x in d for x in ['SWIGGY', 'BUNDL']): return 'SWIGGY'
11    if 'ZOMATO' in d: return 'ZOMATO'
12    if any(x in d for x in ['OLA', 'ANI TECHNOLOGIES']): return 'OLA'
13    if any(x in d for x in ['AMAZON', 'AMZN']): return 'AMAZON'
14    if any(x in d for x in ['RESTAURANT', 'EMPIRE', 'MEGHANA', 'TRUFFLES', 'DINEOUT']): return 'RESTAURANT'
15    if 'ZEPTO' in d: return 'ZEPTO'
16    if 'UBER' in d: return 'UBER'
17    if any(x in d for x in ['BLINKIT', 'GROFERS']): return 'BLINKIT'
18    if any(x in d for x in ['RAPIDO', 'ROPPEEN']): return 'RAPIDO'
19    if 'FLIPKART' in d or 'FKART' in d: return 'FLIPKART'
20    if 'STARBUCKS' in d: return 'STARBUCKS'
21    if any(x in d for x in ['BMTc', 'TUMMOC']): return 'BMTc'
22    if any(x in d for x in ['THIRD WAVE', 'THIRDWAVE', 'TWC INDIA']): return 'THIRD_WAVE'
23    if any(x in d for x in ['BPCL', 'HP PETROL', 'INDIAN OIL', 'IOC']): return 'FUEL'
24    if any(x in d for x in ['NYKAA', 'FSN E-COMMERCE', 'INNOVATIVE RETAIL']): return 'NYKAA'
25    if any(x in d for x in ['CAFE COFFEE DAY', 'COFFEE DAY GLOBAL', 'CCD']): return 'CCD'
26    if any(x in d for x in ['DMART', 'AVENUE SUPERMARTS']): return 'DMART'
27    if 'MYNTRA' in d: return 'MYNTRA'
28    if any(x in d for x in ['BESCOM', 'BANGALORE ELEC', 'BWSSB', 'AIRTEL', 'JIO', 'VODAFONE', 'VI']): return 'UTILITIES'
29    if any(x in d for x in ['NETFLIX', 'HOTSTAR', 'STAR INDIA', 'SPOTIFY']): return 'SUBSCRIPTION'
30    if 'BIGBASKET' in d: return 'BIGBASKET'
31    if any(x in d for x in ['ZERODHA', 'GROWW', 'NEXTBILLION']): return 'INVESTMENT'
32    if any(x in d for x in ['BOOKMYSHOW', 'BMS MOVIE', 'BIGTREE']): return 'BOOKMYSHOW'
33    if 'RENT-LANDLORD' in d: return 'RENT'
34    if 'SALARY' in d: return 'SALARY'
35    if 'ATM' in d: return 'CASH'
36    if any(('UPI-'+name) in d for name in ['AMAN', 'ANKIT', 'KARAN', 'NEHA', 'PRIYA', 'SNEHA', 'VIKAS']):
37        return 'P2P'
38    return 'OTHER'
39
40 # Classify each UNIQUE description once (283 of them), then map back onto
41 # all 1,310 rows -- avoids re-running the same if/elif chain on repeated
42 # descriptions like "BHIM SWIGGY" that appear dozens of times.
43 _unique_lookup = {d: extract_vendor(d) for d in df['Description'].unique()}
44 df['vendor_clean'] = df['Description'].map(_unique_lookup)

```

Category Tagger (Feature 3)

```

1 # Feature 3: Category Tagger
2 # Covers every label vendor_clean can produce. 'SALARY' -> 'Income' (it's a
3 # credit, not spend -- must never sit inside a spend category, or it will
4 # inflate that category's total and distort savings rate). 'RENT' -> 'Housing'
5 # (no slot in the original 12 categories, given its own bucket instead of
6 # silently folding into 'Other').
7 cat_map = {
8     'SWIGGY': 'Food Delivery', 'ZOMATO': 'Food Delivery', 'RESTAURANT': 'Restaurants',
9     'BLINKIT': 'Quick Commerce', 'ZEPTO': 'Quick Commerce',

```

```

10 'BIGBASKET':'Groceries', 'DMART':'Groceries',
11 'AMAZON':'E-commerce', 'FLIPKART':'E-commerce', 'MYNTRA':'E-commerce', 'NYKAA':'E-commerce',
12 'UBER':'Transport', 'OLA':'Transport', 'RAPIDO':'Transport', 'BMTCL':'Transport',
13 'FUEL':'Fuel',
14 'STARBUCKS':'Cafe', 'CCD':'Cafe', 'THIRD_WAVE':'Cafe',
15 'SUBSCRIPTION':'Subscriptions',
16 'UTILITIES':'Utilities',
17 'BOOKMYSHOW':'Entertainment',
18 'INVESTMENT':'Investments',
19 'P2P':'Personal Transfer',
20 'CASH':'Cash Withdrawal',
21 'RENT':'Housing',
22 'SALARY':'Income',
23 'INSTAMART':'Quick Commerce'
24 }
25 df['category'] = df['vendor_clean'].map(cat_map).fillna('Other')
26
27 print("Category Distribution:")
28 print(df['category'].value_counts())

```

Category Distribution:

category	
Food Delivery	297
Transport	250
Quick Commerce	193
E-commerce	180
Cafe	99
Restaurants	73
Utilities	40
Groceries	33
Subscriptions	31
Fuel	28
Investments	23
Personal Transfer	18
Cash Withdrawal	17
Entertainment	13
Income	6
Housing	6
Other	3

Name: count, dtype: int64

✓ Spending Overview (Feature 4)

```

1 # Feature 4: Spending Overview
2 debits = df[df['Type'] == 'Debit'] # capital D -- matches how F1 standardized Type
3 credits = df[df['Type'] == 'Credit'] # capital C
4
5 total_debits = debits['Amount'].sum()
6 total_credits = credits['Amount'].sum()
7 net_savings = total_credits - total_debits
8 savings_rate = (net_savings / total_credits * 100) if total_credits > 0 else 0
9
10 print("=== SPENDING OVERVIEW ===")
11 print(f"Total Credits: ₹{total_credits:,.0f}")
12 print(f"Total Debits: ₹{total_debits:,.0f}")
13 print(f"Net Savings: ₹{net_savings:,.0f}")
14 print(f"Savings Rate: {savings_rate:.1f}%")
15 print(f"Total Transactions: {len(df)}")
16 print("\nTop Categories by Spend:")
17 print(debits.groupby('category')['Amount'].sum().sort_values(ascending=False).head(8))

```

=== SPENDING OVERVIEW ===
 Total Credits: ₹509,774
 Total Debits: ₹1,678,901
 Net Savings: ₹-1,169,127
 Savings Rate: -229.3%
 Total Transactions: 1310

Top Categories by Spend:

category	
E-commerce	613076.0
Investments	248160.0
Food Delivery	129054.0
Restaurants	117737.0
Housing	108000.0
Quick Commerce	95667.0
Fuel	89303.0
Transport	57474.0

Name: Amount, dtype: float64

Monthly Trend (Feature 5)

```

1 # Feature 5: Monthly Trend Analysis
2 df['Month'] = df['Date'].dt.month # numeric, sorts correctly
3 debits = df[df['Type'] == 'Debit'].copy()
4 debits['Month'] = debits['Date'].dt.month
5
6 monthly_spend = debits.pivot_table(
7     values='Amount',
8     index='category',
9     columns='Month',
10    aggfunc='sum',
11    fill_value=0
12 ).rename(columns=lambda m: pd.Timestamp(2024, m, 1).strftime('%b')) # relabel Jan..Jun, still in order
13
14 print("Monthly Spending Trend (Top Categories):")
15 top_cats = debits.groupby('category')['Amount'].sum().nlargest(5).index
16 print(monthly_spend.loc[top_cats])

```

Monthly Spending Trend (Top Categories):						
Month	Jan	Feb	Mar	Apr	May	Jun
category						
E-commerce	98623.0	95065.0	108981.0	74742.0	97632.0	138033.0
Investments	38432.0	15000.0	68644.0	54126.0	48628.0	23330.0
Food Delivery	20890.0	21452.0	20850.0	23054.0	22167.0	20641.0
Restaurants	16320.0	21772.0	28313.0	7711.0	22286.0	21335.0
Housing	18000.0	18000.0	18000.0	18000.0	18000.0	18000.0

Time-of-Day Patterns (Feature 6)

```

1 # Feature 6: Time-of-Day Patterns
2 debits = df[df['Type'] == 'Debit'].copy() # capital D
3
4 debits['Hour'] = pd.to_datetime(debits['Time'], format='%H:%M', errors='coerce').dt.hour
5
6 food_late = debits[(debits['category'] == 'Food Delivery') &
7                    (debits['Hour'].notna()) &
8                    ((debits['Hour'] >= 21) | (debits['Hour'] <= 2))]
9
10 denom = len(debits[debits['category'] == 'Food Delivery'])
11 late_pct = (len(food_late) / denom) * 100 if denom > 0 else 0
12
13 print(f"Late Night Food Delivery (9PM-2AM): {late_pct:.1f}%")

```

Late Night Food Delivery (9PM-2AM): 21.2%

Anomaly Detection (Feature 7)

```

1 # ai assested : learing the z score type and implementation
2
3 # Feature 7: Anomaly Detection
4 debits = df[df['Type'] == 'Debit'].copy() # capital D
5
6 debits['category_mean'] = debits.groupby('category')['Amount'].transform('mean')
7 debits['category_std'] = debits.groupby('category')['Amount'].transform('std')
8 debits['z_score'] = (debits['Amount'] - debits['category_mean']) / debits['category_std']
9
10 anomalies = debits[debits['z_score'] > 2].sort_values('z_score', ascending=False)
11 print("Top Anomalies:")
12 print(anomalies[['Date', 'vendor_clean', 'category', 'Amount', 'z_score']].head(8))

```

Top Anomalies:					
	Date	vendor_clean	category	Amount	z_score
1280	2024-06-26	AMAZON	E-commerce	22008.0	4.182130
266	2024-02-07	AMAZON	E-commerce	21986.0	4.177184
409	2024-02-26	RESTAURANT	Restaurants	8383.0	3.884639
470	2024-03-05	AMAZON	E-commerce	19917.0	3.712029
1253	2024-06-22	RESTAURANT	Restaurants	7935.0	3.627582
646	2024-03-31	RESTAURANT	Restaurants	7931.0	3.625287
459	2024-03-04	RESTAURANT	Restaurants	7441.0	3.344131
445	2024-03-02	AMAZON	E-commerce	18273.0	3.342423

Archetypes (Feature 8)

```

1 # ai assessted : design the architype what paramiter shuold i used and approch and formulation.
2
3 # Feature 8: Spending Archetype Detection
4
5 # 'hour' column pehle bana lo, kyunki abhi tak nahi hai
6 df['hour'] = df['Time'].str[:2].astype(int)
7
8 debits = df[df['Type'] == 'Debit'].copy()
9 credits = df[df['Type'] == 'Credit'].copy()
10
11 # exclude non-consumption categories from % calculations
12 spend = debits[~debits['category'].isin(['Personal Transfer', 'Cash Withdrawal'])]
13 total_spend = spend['Amount'].sum()
14
15 def pct(cat_list):
16     return spend[spend['category'].isin(cat_list)]['Amount'].sum() / total_spend * 100
17
18 food_pct = pct(['Food Delivery', 'Restaurants', 'Cafe'])
19 qc_pct = pct(['Quick Commerce'])
20 ecom_pct = pct(['E-commerce'])
21 inv_pct = pct(['Investments'])
22 transport_pct = pct(['Transport'])
23
24 total_credits = credits['Amount'].sum()
25 total_debits = debits['Amount'].sum()
26 savings_rate = (total_credits - total_debits) / total_credits * 100
27
28 def late_night_snacker_pct():
29     fd = debits[debits['category'] == 'Food Delivery']
30     if len(fd) == 0:
31         return 0
32     late = fd[(fd['hour'] >= 21) | (fd['hour'] <= 2)]
33     return len(late) / len(fd) * 100
34
35 late_pct = late_night_snacker_pct()
36 sub_vendor_count = debits[debits['category'] == 'Subscriptions']['vendor_clean'].nunique()
37
38 def detect_foodie(): return food_pct > 25
39 def detect_quick_commerce(): return qc_pct > 15
40 def detect_shopaholic(): return ecom_pct > 15
41 def detect_investor(): return inv_pct > 15
42 def detect_late_night(): return late_pct > 50
43 def detect_cab_commuter(): return transport_pct > 10
44 def detect_subscription_lover(): return sub_vendor_count >= 5
45 def detect_yolo(): return savings_rate < 10
46 def detect_disciplined_saver(): return savings_rate > 40
47
48 archetypes = []
49 if detect_foodie(): archetypes.append(f"THE FOODIE ({food_pct:.1f}% on food)")
50 if detect_quick_commerce(): archetypes.append(f"THE QUICK COMMERCE JUNKIE ({qc_pct:.1f}%)")
51 if detect_shopaholic(): archetypes.append(f"THE SHOPAHOLIC ({ecom_pct:.1f}%)")
52 if detect_investor(): archetypes.append(f"THE INVESTOR ({inv_pct:.1f}%)")
53 if detect_late_night(): archetypes.append(f"THE LATE-NIGHT SNACKER ({late_pct:.1f}% food after 9PM)")
54 if detect_cab_commuter(): archetypes.append(f"THE CAB COMMUTER ({transport_pct:.1f}%)")
55 if detect_subscription_lover(): archetypes.append(f"THE SUBSCRIPTION LOVER ({sub_vendor_count} active subs)")
56 if detect_yolo(): archetypes.append(f"THE YOLO SPENDER (savings rate {savings_rate:.1f}%)")
57 if detect_disciplined_saver(): archetypes.append(f"THE DISCIPLINED SAVER (savings rate {savings_rate:.1f}%)")
58
59 print("=== SPENDING ARCHETYPES ===")
60 for a in archetypes:
61     print(f"-> {a}")
62
63 print(f"\n(Supporting Stats - Food: {food_pct:.1f}%, Quick Commerce: {qc_pct:.1f}%, "
64       f"E-commerce: {ecom_pct:.1f}%, Investments: {inv_pct:.1f}%, "
65       f"Late-night: {late_pct:.1f}%, Savings rate: {savings_rate:.1f}%)")

```

=== SPENDING ARCHETYPES ===

-> THE SHOPAHOLIC (38.1%)

-> THE INVESTOR (15.4%)

-> THE YOLO SPENDER (savings rate -229.3%)

(Supporting Stats - Food: 17.3%, Quick Commerce: 5.9%, E-commerce: 38.1%, Investments: 15.4%, Late-night: 21.2%, Savings rat

✓ FINAL SPENDDNA REPORT - Beautiful & Statistical Summary

```

1 # ai assessted : help in formating
2
3 # Final Report
4
5 # debits = df[df['Type'] == 'Debit'].copy()

```

```

5 debits = df[df['Type'] == 'Debit'].copy()
6 credits = df[df['Type'] == 'Credit'].copy()
7
8 total_credits = credits['Amount'].sum()
9 total_debits = debits['Amount'].sum()
10 net_change = total_credits - total_debits
11 savings_rate = net_change / total_credits * 100
12
13 # consumption-only spend (excludes P2P transfers & cash withdrawals)
14 spend = debits[~debits['category'].isin(['Personal Transfer', 'Cash Withdrawal'])]
15 total_spend = spend['Amount'].sum()
16
17 print("=" * 70)
18 print(" SpendDNA REPORT – RAHUL SHARMA")
19 print(f" 6 months • {len(df):,} transactions • Jan to Jun 2024")
20 print("=" * 70)
21
22 # ----- EXECUTIVE SUMMARY -----
23 print("\n EXECUTIVE SUMMARY")
24 print(f" Total credits : Rs. {total_credits:,.0f}")
25 print(f" Total debits : Rs. {total_debits:,.0f}")
26 print(f" Net change : Rs. {net_change:,.0f} ({'overspending' if net_change < 0 else 'saving'})")
27 tag = "🔥 BURNING SAVINGS" if savings_rate < 0 else ("🟢 HEALTHY" if savings_rate > 20 else "⚠️ TIGHT")
28 print(f" Savings rate : {savings_rate:.1f}% {tag}")
29 print(f" Transactions : {len(df):,}")
30 print(f" Unique vendors : {df['vendor_clean'].nunique()}")
31
32 # ----- TOP CATEGORIES -----
33 print("\n TOP CATEGORIES (% of consumption spend)")
34 cat_spend = spend.groupby('category')['Amount'].sum().sort_values(ascending=False)
35 for cat, amt in cat_spend.head(7).items():
36     pct = amt / total_spend * 100
37     bars = "#" * int(pct / 3)
38     print(f" {cat:<16} {bars:<24} {pct:5.1f}% Rs. {amt:>10,.0f}")
39
40 # ----- TOP VENDORS -----
41 print("\n TOP VENDORS")
42 vend = spend.groupby('vendor_clean').agg(total=('Amount', 'sum'), orders=('Amount', 'count'))
43 vend = vend.sort_values('total', ascending=False).head(5)
44 for name, row in vend.iterrows():
45     print(f" {name:<15} Rs. {row['total']:>9,.0f} ({int(row['orders']):>3} orders)")
46
47 # ----- TIME-OF-DAY PATTERNS -----
48 print("\n TIME-OF-DAY PATTERNS")
49 fd = debits[debits['category'] == 'Food Delivery']
50 late_pct = ((fd['hour'] >= 21) | (fd['hour'] <= 2)).mean() * 100 if len(fd) else 0
51 print(f" Food Delivery peaks : 9 PM – 2 AM ({late_pct:.1f}% of orders)")
52
53 cafe = debits[debits['category'] == 'Cafe']
54 morning_pct = ((cafe['hour'] >= 8) & (cafe['hour'] <= 11)).mean() * 100 if len(cafe) else 0
55 print(f" Cafe peaks : 8 AM – 11 AM ({morning_pct:.1f}% of orders – morning coffee runs)")
56
57 qc = debits[debits['category'] == 'Quick Commerce']
58 qc_evening = ((qc['hour'] >= 18) & (qc['hour'] <= 22)).mean() * 100 if len(qc) else 0
59 print(f" Quick Commerce : {qc_evening:.1f}% between 6–10 PM, rest spread through the day")
60
61 # ----- MONTHLY TREND (top category) -----
62 top_cat = cat_spend.index[0]
63 print(f"\n MONTHLY TREND ({top_cat})")
64 month_names = {1:'Jan', 2:'Feb', 3:'Mar', 4:'Apr', 5:'May', 6:'Jun'}
65 monthly = spend[spend['category'] == top_cat].groupby('Month')['Amount'].sum()
66 max_val = monthly.max()
67 for m in range(1, 7):
68     val = monthly.get(m, 0)
69     bars = "#" * int(val / max_val * 20) if max_val else ""
70     print(f" {month_names[m]} Rs. {val:>9,.0f} {bars}")
71
72 # ----- ANOMALIES -----
73 print("\n TOP ANOMALIES (z-score > 2, within category)")
74 debits['cat_mean'] = debits.groupby('category')['Amount'].transform('mean')
75 debits['cat_std'] = debits.groupby('category')['Amount'].transform('std')
76 debits['z_score'] = (debits['Amount'] - debits['cat_mean']) / debits['cat_std']
77 anomalies = debits[debits['z_score'] > 2].sort_values('z_score', ascending=False)
78 for _, r in anomalies.head(5).iterrows():
79     print(f" {r['Date'].strftime('%d %b')} - {r['vendor_clean']:<12} Rs. {r['Amount']:>8,.0f} (z={r['z_score']:.1f})")
80
81 # ----- ARCHETYPES -----
82 def pct_of(cat_list):
83     return spend[spend['category'].isin(cat_list)]['Amount'].sum() / total_spend * 100
84
85 food_pct = pct_of(['Food Delivery', 'Restaurants', 'Cafe'])
86 qc_pct = pct_of(['Quick Commerce'])
87 ecom_pct = pct_of(['E-commerce'])

```

```

88 inv_pct = pct_of(['Investments'])
89 trans_pct = pct_of(['Transport'])
90 sub_count = debits[debits['category'] == 'Subscriptions']['vendor_clean'].nunique()
91
92 print("\n RAHUL'S SPENDING ARCHETYPES")
93 archetypes = []
94 if food_pct > 25: archetypes.append(f"🍔 THE FOODIE ({food_pct:.1f}% on food)")
95 if qc_pct > 15: archetypes.append(f"🚀 THE QUICK COMMERCE JUNKIE ({qc_pct:.1f}%)")
96 if ecom_pct > 15: archetypes.append(f"🛒 THE SHOPAHOLIC ({ecom_pct:.1f}%)")
97 if inv_pct > 15: archetypes.append(f"📈 THE INVESTOR ({inv_pct:.1f}%)")
98 if late_pct > 50: archetypes.append(f"🌙 THE LATE-NIGHT SNACKER ({late_pct:.1f}% food after 9PM)")
99 if trans_pct > 10: archetypes.append(f"🚗 THE CAB COMMUTER ({trans_pct:.1f}%)")
100 if sub_count >= 5: archetypes.append(f"📺 THE SUBSCRIPTION LOVER ({sub_count} active subs)")
101 if savings_rate < 10: archetypes.append(f"🔥 THE YOLO SPENDER (savings rate {savings_rate:.1f}%)")
102 if savings_rate > 40: archetypes.append(f"💰 THE DISCIPLINED SAVER (savings rate {savings_rate:.1f}%)")
103 for a in archetypes:
104     print(f" -> {a}")
105
106 # ----- KEY INSIGHTS (dynamic, data-specific) -----
107 print("\n" + "=" * 70)
108 print(" KEY INSIGHTS")
109 monthly_burn = abs(net_change) / 6
110 print(f" 1. Rahul is burning through savings at Rs. {monthly_burn:,.0f}/month")
111 print(f" -> Savings rate of {savings_rate:.1f}% is unsustainable long-term.")
112 print(f" 2. {top_cat} is the single biggest category at {cat_spend.iloc[0]/total_spend*100:.1f}% of spend,")
113 print(f" driven by fewer but high-ticket transactions (avg Rs. "
114        f"{spend[spend['category']==top_cat]['Amount'].mean():,.0f}/order).")
115 print(f" 3. {late_pct:.1f}% of food-delivery orders happen after 9 PM,")
116 print(f" suggesting late-working or single-living habits.")
117 print(f" 4. Investments (SIP + Zerodha) sit at {inv_pct:.1f}% - a healthy floor,")
118 print(f" but discretionary categories still dominate the wallet.")
119 print("=" * 70)

```

Total debits : Rs. 1,678,901
 Net change : Rs. -1,169,127 (overspending)
 Savings rate : -229.3% 🔥 BURNING SAVINGS
 Transactions : 1,310
 Unique vendors : 29

TOP CATEGORIES (% of consumption spend)

E-commerce	#####	38.1%	Rs.	613,076
Investments	#####	15.4%	Rs.	248,160
Food Delivery	##	8.0%	Rs.	129,054
Restaurants	##	7.3%	Rs.	117,737
Housing	##	6.7%	Rs.	108,000
Quick Commerce	#	5.9%	Rs.	95,667
Fuel	#	5.6%	Rs.	89,303

TOP VENDORS

AMAZON	Rs.	328,530	(86 orders)
INVESTMENT	Rs.	248,160	(23 orders)
FLIPKART	Rs.	177,510	(47 orders)
RESTAURANT	Rs.	117,737	(73 orders)
RENT	Rs.	108,000	(6 orders)

TIME-OF-DAY PATTERNS

Food Delivery peaks : 9 PM - 2 AM (21.2% of orders)
 Cafe peaks : 8 AM - 11 AM (35.4% of orders - morning coffee runs)
 Quick Commerce : 40.4% between 6-10 PM, rest spread through the day

MONTHLY TREND (E-commerce)

Jan	Rs.	98,623	#####
Feb	Rs.	95,065	#####
Mar	Rs.	108,981	#####
Apr	Rs.	74,742	#####
May	Rs.	97,632	#####
Jun	Rs.	138,033	#####

TOP ANOMALIES (z-score > 2, within category)

26 Jun - AMAZON	Rs.	22,008	(z=4.2)
07 Feb - AMAZON	Rs.	21,986	(z=4.2)
26 Feb - RESTAURANT	Rs.	8,383	(z=3.9)
05 Mar - AMAZON	Rs.	19,917	(z=3.7)
22 Jun - RESTAURANT	Rs.	7,935	(z=3.6)

RAHUL'S SPENDING ARCHETYPES

-> 🛒 THE SHOPAHOLIC (38.1%)
 -> 📈 THE INVESTOR (15.4%)
 -> 🔥 THE YOLO SPENDER (savings rate -229.3%)

```
suggesting late-working or single-living habits.  
4. Investments (SIP + Zerodha) sit at 15.4% – a healthy floor,  
but discretionary categories still dominate the wallet.  
=====
```